Debugging SoC Integration – Detecting a Real Bug

Introduction

This lab exposes you to practical SoC integration errors specifically bit width mismatches and gives hands-on practice in detecting and troubleshooting these bugs using simulation and waveform analysis. You will discover how a subtle wiring error can propagate into simulation artifacts, then learn to identify, diagnose, and resolve such mistakes by observing behaviour in GTKWave.

Objective

- Understand how small integration mistakes (e.g., bit-width mismatches) can cause unexpected behavior.
- Learn how to detect such bugs using Verilator and GTKWave.
- Practice writing modular testbenches and debugging using waveform output.

Procedure to create a Lab project:

Create a project directory `soc_debug_lab`

Navigate to `soc_debug_lab` directory and create the alu, uart_stub, soc_top, testbench files.

Block Diagram :

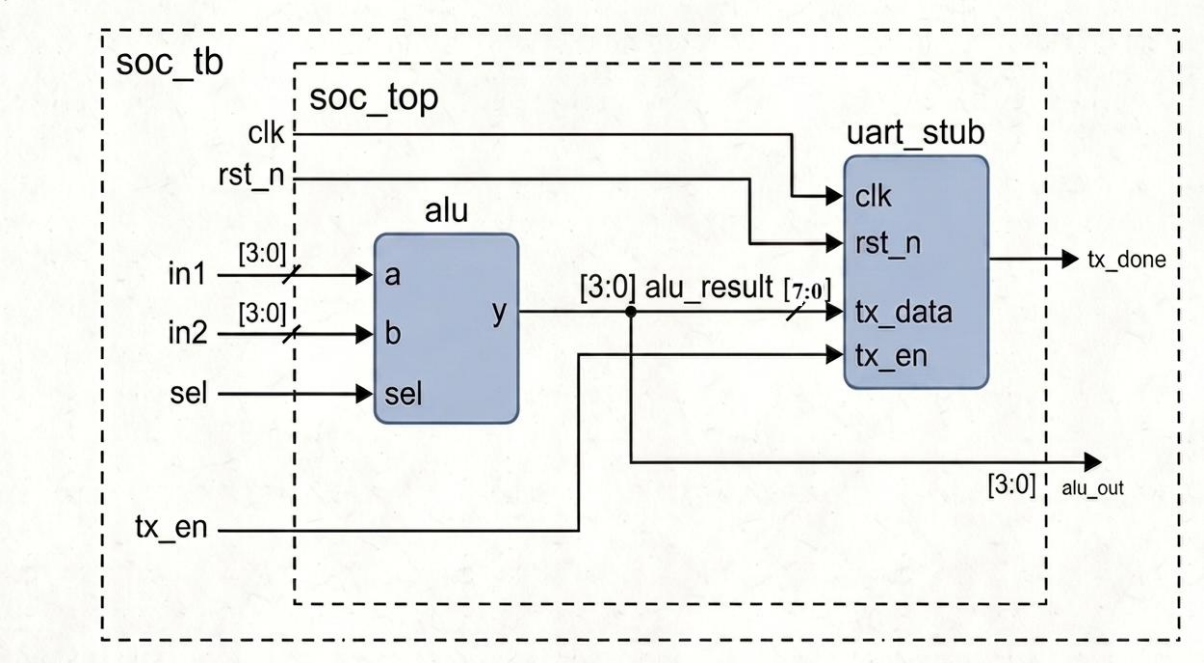


Fig: SoC Integration

Steps to create the lab:

1: ALU:

ALU will performs either AND or ADD on two 4-bit inputs.

Create the file for ALU: `alu.v`

```
// Simple 4-bit ALU that performs bitwise AND and addition
module alu (
  input  [3:0] a,       // 4-bit operand A
  input  [3:0] b,       // 4-bit operand B
  input  sel,          // Operation select: 0 = AND, 1 = ADD
  output reg [3:0] y    // 4-bit ALU output
);
  always @(*) begin
    case (sel)
      1'b0: y = a & b;          // Bitwise AND operation
      1'b1: y = a + b;          // 4-bit addition
      default: y = 4'b0000;
    endcase
  end
endmodule
```

2: UART Stub

Simulates basic UART send behaviour for testbench purposes.

Create a file for UART stub: `uart_stub.v`

```
module uart_stub (
  input clk,                // Clock input
  input rst_n,              // Active-low reset
  input tx_en,              // Transmit enable
  input [7:0] tx_data,      // 8-bit transmit data
  output reg tx_done    );   // Transmit done flag

 always @(posedge clk or negedge rst_n) begin
```

```
    if (!rst_n)
      tx_done <= 0;
    else if (tx_en)
      tx_done <= 1;
    else
      tx_done <= 0;
  end
endmodule
```

3: SoC Integration for Debug Analysis

Create a file for SoC Integration: `soc_top.v`

SoC integration where ALU output is passed to UART.

// Intentional Bug: alu_result (4-bit) passed directly to UART (8-bit input)

```
module soc_top (
  input clk,
  input rst_n,
  input [3:0] in1,
  input [3:0] in2,
  input sel,
  input tx_en,
  output tx_done,
  output [3:0] alu_out
);
  wire [3:0] alu_result;
  // ALU instance for arithmetic/logic operation
  alu alu_dut (
    .a(in1),
    .b(in2),
    .sel(sel),
    .y(alu_result)
  );
```

```
// BUG: tx_data expects 8 bits but alu_result is only 4 bits.
  // The upper 4 bits of tx_data will be undefined or X (unknown) in simulation.
  Uart_stub uart_dut (
    .clk(clk),
    .rst_n(rst_n),
    .tx_en(tx_en),
    .tx_data(alu_result),    // ← Mistake: should be {4'b0000, alu_result}
    .tx_done(tx_done)
  );
  assign alu_out = alu_result;
endmodule
```

- **BUG:** The 4-bit ALU result is directly connected to the UART's 8-bit data input, so upper bits float, producing potentially 'X' (unknown) values during simulation.

4: Create testbench file : soc_tb.v

Stimulates both ADD and AND operations, each time enabling transmit so UART will record outcome.

```
// Testbench that drives stimulus and dumps waveform
module soc_tb;
  reg clk = 0;
  reg rst_n = 0;
  reg [3:0] in1, in2;
  reg sel;
  reg tx_en;
  wire tx_done;
  wire [3:0] alu_out;
  always #5 clk = ~clk;
```

```
// Instantiate the DUT (SoC with Bug)
  soc_top uut (
    .clk(clk),
    .rst_n(rst_n),
    .in1(in1),
    .in2(in2),
    .sel(sel),
    .tx_en(tx_en),
    .tx_done(tx_done),
  .alu_out(alu_out)
  );
  initial begin
    $dumpfile("soc_debug.vcd");
    $dumpvars();
    // Apply reset
    #10 rst_n = 1;
    // Test 1: ADD 4 + 3 = 7, expect UART to transmit 8'b00000111
    in1 = 4; in2 = 3; sel = 1; tx_en = 1;
    #10 tx_en = 0;
  // Test 2: AND 6 & 5 = 4, expect UART to transmit 8'b00000100
    #20 in1 = 6; in2 = 5; sel = 0; tx_en = 1;
    #10 tx_en = 0;
    #40 $finish;
  end
endmodule
```

5) Simulation Script:

a) Create “run.sh”

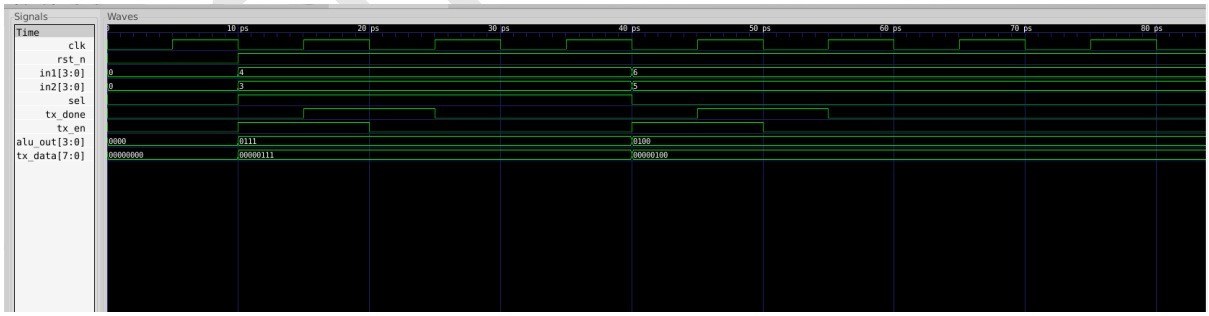
```
#!/bin/bash

# Step 1: Run Verilator to compile and generate simulation files
verilator --binary -j 0 -Wall \
  alu.v uart_stub.v soc_top.v soc_tb.v \
  --top soc_tb --timing --CFLAGS "-std=c++20" --trace

# Step 2: Navigate to the object directory
cd obj_dir || { echo "obj_dir not found"; exit 1; }

# Step 3: Compile the simulation binary
make -f Vsoc_tb.mk Vsoc_tb || { echo "Compilation failed";
exit 1; }

# Step 4: Run the simulation
./Vsoc_tb || { echo "Simulation failed"; exit 1; }

# Step 5: Open the waveform in GTKWave
gtkwave soc_debug.vcd
```

b) Make the script executable:

```
chmod +x run.sh
```

c) Run the script to perform compilation, simulation, and open the waveform.

GTKWave Waveform



Signal	Expected	Observed (Bug)
alu_out	Correct (4 bits)	Correct
tx_done	Toggles with tx_en	Toggles
tx_data	8-bit padded	Upper 4 bits = 'X' (undefined) Upper 4 bits = 0 (for verilator)

- **Bug Manifestation:** In the wave viewer, the UART's tx_data signal's top 4 bits will show as XXXX whenever transmitting (should be zero-padded).
- This can create mismatches or confusion, and would not synthesize correctly to hardware.
- **Note :** Verilator supports 2 states (either 0 or 1) , output is already 0 padded in case of verilator output in waveform.

Debugging and Fix

How to Fix:
In soc_top.v, change the connection for the UART's tx_data from:
`.tx_data(alu_result)` to `.tx_data({4'b0000, alu_result})` *// Zero-pad upper 4 bits*

This concatenates four zeros above the 4-bit result to produce a clean 8-bit aligned word.

Key Learnings

Concept	Learning Outcome
Signal Width Matching	Always ensure port widths match at IP block boundaries
Modular Design	Isolate logic into blocks for easier integration/testing
Debug with GTKWave	Visually trace signal width and logic errors
Practical Debugging	Experience a common SoC integration bug's result
Verilator Workflow	Simulate, observe, fix

Conclusion

This lab demonstrates how small integration mistakes, such as mismatched signal widths, can introduce subtle yet critical bugs into SoC designs. Through simulation and waveform review, you observed undefined simulation values and their consequences. Learning to identify and fix such issues by careful observation and modular testbench design is an indispensable skill for digital hardware engineers. The modular approach not only makes designs easier to test and debug but also reduces the risk of integration errors when assembling complex SoCs. Employing best practices in signal width alignment, simulation, and visual debug tools will result in more reliable and robust digital designs.